EXPERIMENT 10

Performance Optimization in Zoho Creator

Name: Jobeson J | **Register Number:** 240701217
Subject: Low Code Application Development using Zoho Creator
Project: Bank Loan Application & Pre-Screening Management

Aim

To apply performance optimisation techniques to the Loan Application, in particular efficient record retrieval and workflow processing, and to compare an inefficient script with an optimised one.

Introduction

A low-code platform hides most of the infrastructure, but the way a workflow is written still decides how much data it reads and how much work it repeats. Retrieving more records than needed, or processing records that are not relevant, costs time on every run. With a handful of test records the difference is invisible; with thousands of loan applications and a schedule that runs every day, it becomes important.

Application Used

The Loan_Application_Form and the Daily_Pending_Document_Reminder_Schedule of Experiment 6 are used. The schedule must find applications with pending documents and send each a reminder.

Optimization Techniques

Table 10.1: Optimisation techniques

Technique	What it means	Applied to
Fetch with a criteria	Ask the database only for matching records instead of reading all and filtering in the script	Pending-document query
Avoid fetch inside loops	Do not run a new fetch for every record of an outer loop; fetch once and reuse	Reminder script
Use count instead of fetch	When only a number is needed, use count rather than loading records	Dashboard figures
Limit the range	Fetch in pages (range) for large result sets	Batch EMI reminders
Batch processing	Process records in controlled batches so a run does not time out	Bulk_EMI_Reminder_Batch
Update only what changed	Trigger report workflows only when specific fields change	OnEdit_Recalculate_EMI_Workflow
Lean forms and reports	Show only needed columns and use default filters	All_Loan_Applications_List

Inefficient versus Optimised Script

```
// Inefficient: reads every application, then tests each one inside the loop
all_apps = Loan_Application_Form[ID != 0];
for each rec in all_apps
{
    if (rec.Documents_Complete == false)
    {
        // an extra fetch for every record
        officer = Officers[Officer_ID == rec.Assigned_Officer_ID];
        sendmail [ from: zoho.adminuserid to: rec.Email_Address subject: "Documents pending" message: "Plea
    }
}
```

Code 10.1 — Inefficient approach

```
// Optimised: the criteria selects only pending applications; no fetch inside the loop
pending = Loan_Application_Form[Documents_Complete == false && Reminder_Count < 5];
for each rec in pending
{
    sendmail [ from: zoho.adminuserid to: rec.Email_Address subject: "Documents pending" message: "Please u
    rec.Reminder_Count = rec.Reminder_Count + 1;
}
```

Code 10.2 — Optimised approach

- Only matching records are read, so the number of records processed falls to those that need a reminder.
- The per-record fetch of the officer is removed; officer details are already stored or looked up once.
- The Reminder_Count limit stops repeated reminders and so also reduces the number of e-mails sent.

Measurement Plan

To measure the difference, record the time before and after each script with **zoho.currenttime** in an **info** statement, using the same test data for both versions. The table below is to be filled with the readings taken during the experiment.

Table 10.2: Measured times (to be completed)

Test data size	Inefficient script (seconds)	Optimised script (seconds)	Records processed (inefficient / optimised)
10 records			
100 records			
500 records			

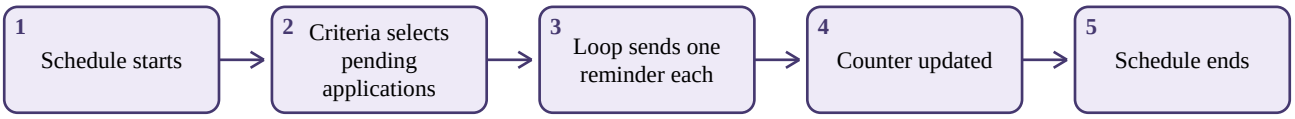


Fig 10.1 — Optimised workflow (schematic drawn from the specification; not a screenshot)

Output

SCREENSHOT TO ATTACH

Both versions of the script in the Deluge editor

Fig 10.2 — Inefficient and optimised scripts

SCREENSHOT TO ATTACH

Execution log showing the info statements with the times for each version

Fig 10.3 — Execution log with timings

Result

The pending-document reminder was written in an inefficient and an optimised form, and a method for comparing their running times was recorded.

Conclusion

The simplest optimisations are the most effective: select only the records needed, never fetch inside a loop, count instead of fetching, and process large sets in batches. These habits keep the Loan Application responsive as the number of applications and users grows.